

ISLAMIC UNIVERSITY OF TECHNOLOGY (IUT)
ORGANISATION OF ISLAMIC COOPERATION (OIC)
Department of Computer Science and Engineering (CSE)

Team-4

UniConnect

Final Report for SWE 4404 Software Project Lab II

Supervisor : **Ajwad Abrar Mostofa, Junior Lecturer, CSE**

Repository : https://github.com/aymannondiny/SPL-II_UniConnect

Team member	Student ID
Maliha Tasnim Khan	230042127
Sayma Tasnim	230042139
Ayman Binta Altaf Nondiny	230042141
Saika Sarara	230042159

Submission Date : **1 November 2026**

Table of Contents

Table of Contents 2
1 Executive Summary 4
CO6 Self Assessment 4
2 Problem Domain and Background 4
Application Domain and Users 4
Underlying Engineering Concepts 5
CO1 Self Assessment : Fundamental concepts behind UniConnect 5
CO1 Self Assessment : Specialist knowledge needed for this domain 6
3 Literature and Existing Solutions 6
CO2 Self Assessment 7
4 Requirements 7
Stakeholders and Baselines 7
Requirement Changes 8
Context and sustainability considerations 8
Feedback and response 8
CO2 Self Assessment 9
5 System Design 9
Architecture 9
Data Model and Database Schema 9
Design Alternatives 11
Traceability to the SRS 12
Design session feedback 12
CO3 Self Assessment 12
6 Implementation 12
Project Structure 12
Communication and Data Boundaries 12
Division of Classes and Methods 14
Implemented Features 14
Evolution and Refactoring 16
CO1 and CO3 Self Assessment 16
7 Tools Version Control Testing and Continuous Integration 17
Technology and Tool Choices 17
Version Control 17
Testing and Continuous Integration 18
CO4 Self Assessment 18
8 Professional Ethical and Legal Considerations 18
Implemented Controls 18

Outstanding Governance 19
CO4 Self Assessment 19
9 Project Management and Feasibility 19
Timeline Evidence 19
Feasibility 20
CO7 Self Assessment 20
10 Teamwork and Individual Contributions 20
CO5 Self Assessment 21
11 Results Limitations and Future Work 21
Final Requirement Status 21
Known Limitations 22
Future Work 22
CO6 Self Assessment 22
12 Glossary and References 22
Glossary 22
CO6 Self Assessment 23
References 23

1 Executive Summary

Our project UniConnect is built for IUT students, alumni, club operators, and system administrators. Here we have addressed a practical problem: people spread academic collaboration, campus announcements, mentorship, career opportunities, and networking across unrelated messaging groups, social networks, email, and personal contacts. That fragmentation makes information difficult to find and limits the university's control over identity, access, and moderation.

We have combined a Flutter web and Android client, a Spring Boot backend, and a PostgreSQL database. Members register with a university email, verify an emailed six-digit code, create role-specific profiles, find people, manage connections, form project teams, join mentorship sessions, chat directly, browse career posts, receive notifications, and interact with club announcements and events. Administrators manage accounts, clubs, requests, reports, and moderation evidence.

The implementation is organized by feature and added with backend and Flutter tests. Our GitHub Actions workflow runs verification and builds web and Android artifacts. We need a linked current run log before we can claim exact passing totals. We completed the principal academic workflows, but we have not completed global search, full account anonymization, a dedicated administrator-warning workflow, comprehensive accessibility and security audits, or production operations.

CO6 Self Assessment

Our project can easily be understood by any non-technical audience who finds the need of an integrated media for community and job searching. We have explained the problem in plain language and defined the technical terms in Section 12, and connected our major claims to requirements, source files, commits, and test evidence in Sections 4 to 7.

2 Problem Domain and Background

Application Domain and Users

We designed UniConnect for a university community. Our proposal identifies students, alumni, club administrators, and system administrators as the principal users. Students need collaborators, mentorship, campus information, and career access. Alumni share professional experience and opportunities. Club operators publish content under assigned authority, while system administrators manage eligibility, club lifecycles, and content reports.

We collected initial requirements through two semi-structured student interviews and by observing campus communication, club recruitment, and project-team formation. These sources helped us frame the problem, but they do not represent the entire university population. We therefore treat them as formative stakeholder input.

Underlying Engineering Concepts

Concept	Use in UniConnect
Object-oriented design	Feature services, domain entities, injected repositories, and DTO boundaries.
Relational modelling	Users, roles, assignments, connections, projects, applications, messages, mentorship occurrences, reports, and audit evidence are represented by constrained relations.
Data structures and algorithms	Sets and maps support validation and aggregation; database ordering and weighted skill matching support recommendations; pagination bounds result sets.
Transactions and concurrency	Sensitive account, verification, enrollment, moderation, and application transitions use transactions, locks, and database constraints.
Security engineering	BCrypt password hashing, database-backed sessions, revocation, scoped authorization, privacy checks, and non-disclosing public responses.

CO1 Self Assessment : Fundamental concepts behind UniConnect

We used **object-oriented programming** to separate responsibilities. Flutter views show information and collect input; backend controllers receive requests; services apply business rules; and repositories read and write data. For example, our connection service manages connection actions within a transaction.

We use **lists, sets, and maps** for different jobs. Lists hold ordered results such as mentor or project candidates. Sets prevent duplicates and identify required skills or members to exclude. Maps let us look up relationships and skill names by ID without repeatedly searching a list. Our project teammate service uses these structures when it prepares recommendations.

We use a **relational database** because accounts, profiles, clubs, connections, projects, mentorship sessions, and messages have defined relationships. Foreign keys and uniqueness rules help protect those relationships; queries, indexes, and pagination help us retrieve relevant records. Transactions keep multi-step actions consistent—for example, accepting a connection or enrolling in a mentorship session must not leave only half the change saved.

Our main **design concept** is a feature-based modular monolith. We group code by capability, such as authentication, clubs, projects, and mentorship, and then divide each feature into API, business-logic, and persistence responsibilities. Data transfer objects define what the Flutter client exchanges with the backend. This structure helps us change a feature while keeping authorization and data rules on the server.

CO1 Self Assessment : Specialist knowledge needed for this domain

- **University identity and authorization:** We must verify university membership and distinguish students, alumni, club operators, and system administrators. A person's role alone is not enough: club actions also depend on a current assignment and context. These checks matter because UniConnect contains restricted campus and administrative functions.
- **Privacy-aware networking:** Members need to discover collaborators without exposing every profile detail to everyone. We apply visibility and connection rules in backend responses, so the client does not receive private information merely to hide it on screen.
- **Matching and recommendation:** Project recruitment needs more than a name search. We compare required and optional skills, interests, and connections to rank candidates; the recommendation scorer isolates that logic.
- **Mentorship scheduling:** Students and alumni can participate in peer mentorship, where recurring meetings, limited capacity, enrollment, cancellation, and history must remain consistent. Our mentorship slot service handles these domain rules.
- **Secure communication:** Chat needs authenticated participants, message history, delivery or read state, and timely updates. We combine stored REST operations with WebSocket events while enforcing access on the backend.

These specialist rules are necessary because UniConnect serves a defined university community with academic collaboration, delegated club authority, and personal member data; not just a general social feed.

3 Literature and Existing Solutions

We studied WhatsApp, Facebook, email, and LinkedIn as reference points in our proposal. WhatsApp Communities offers group coordination and announcements; LinkedIn offers professional profiles, networking, and career discovery. We combined familiar interaction patterns with IUT-domain admission, academic profiles, scoped club authority, project recruitment, peer mentorship, and institutional moderation.
We compared documented functions rather than running a controlled competitor study. Product capabilities may change, and we did not commit a reproducible competitor test dataset. We adapted feeds, directories, messaging, and career-board patterns to one university context without claiming that other products lack every similar feature.

Reference	Relevant capability	UniConnect distinction
WhatsApp Communities	Private group communication, announcements, and community events	Structured academic profiles, project and mentorship workflows, scoped administration, and persistent report evidence.
LinkedIn	Professional networking, profiles, job search, and filters	University-only identity, current-student academic context, campus clubs, project recruitment, and peer mentorship.
Email	Direct asynchronous communication	Discoverable structured content, workflows, authorization, status, notifications, and searchable records.

CO2 Self Assessment

We identified general-purpose alternatives in the SRS and limited our comparison to documented capabilities. Our evidence shows how UniConnect differs in its university-specific combination of functions; it does not prove that competitors lack every comparable function.

4 Requirements

Stakeholders and Baselines

We used the submitted requirements workbook and combined SRS and design report as our academic baseline. Our maintained implementation workbook lists 90 unique functional-requirement identifiers and 93 user-story rows. When older wording differs from the application, we trace the current behavior through source code, migrations, API contracts, and tests.

Stakeholder need	Requirement groups	Implementation response
Trusted university access	FR 1 to 5	University-domain registration, numeric-code verification and recovery, persistent sessions, roles, and contexts.
Discover people and collaborate	FR 6 to 9 and FR 25 to 39	Profiles, privacy, people discovery, connections, project posts,

		recommendations, applications, and membership.
Operate campus clubs	FR 10 to 24 and FR 70 to 71	Club creation, assignments, scoped contexts, announcements, events, suspension, archive, and retained history.
Mentorship and career access	FR 40 to 50 and FR 56 to 61	Mentor eligibility, finite occurrences, capacity, enrollment history, alumni career posts, filters, saves, and closure.
Communication and oversight	FR 51 to 55 and FR 63 to 80	Chat, notifications, account moderation, content reports, evidence validation, and conversation deletion.

Requirement Changes

We replaced the proposed 24-hour verification link with an emailed six-digit code, a ten-minute expiry, five failed attempts, resend controls, and single-use activation.
We changed password recovery from a link to a numeric-code flow.
We introduced explicit archive, restore, and retained-removal lifecycles for clubs.
We have not completed global search, full anonymization, or a dedicated administrator-warning workflow; we only partially resolve notification targets.

Context and sustainability considerations

We considered accessibility, privacy, and limited bandwidth for our university audience. We shared one client codebase across web and Android and bounded result pages to reduce duplicate work and unnecessary transfers. We have not measured energy use or completed an accessibility or stakeholder impact assessment, so we treat these as design considerations.

Feedback and response

Our proposal records feedback that students should be able to search for teammates by skill before creating a project post. We added direct skill-based discovery to the proposed workflow, and the final repository contains discovery and candidate-search paths. In the SRS session, we learned to collect stakeholder requirements and filter our feature scope. After stakeholder interviews, we opened peer mentorship to alumni, improved the career board's relevant information, and refined the club activity workspace. We do not have a signed session feedback log in the repository.

CO2 Self Assessment

We separated functional requirements from acceptance criteria in the workbooks and mapped requirement groups to stakeholder needs above. We can trace the implementation at the group level, but we cannot claim formal stakeholder approval for every changed contract without a signed record.

5 System Design

Architecture

We built a three-tier modular monolith. We placed presentation and client state in Flutter; REST, authenticated chat, security, business rules, and DTO mapping in Spring Boot; and durable records in PostgreSQL. We use Flyway migrations for schema changes and SMTP for verification and recovery codes.

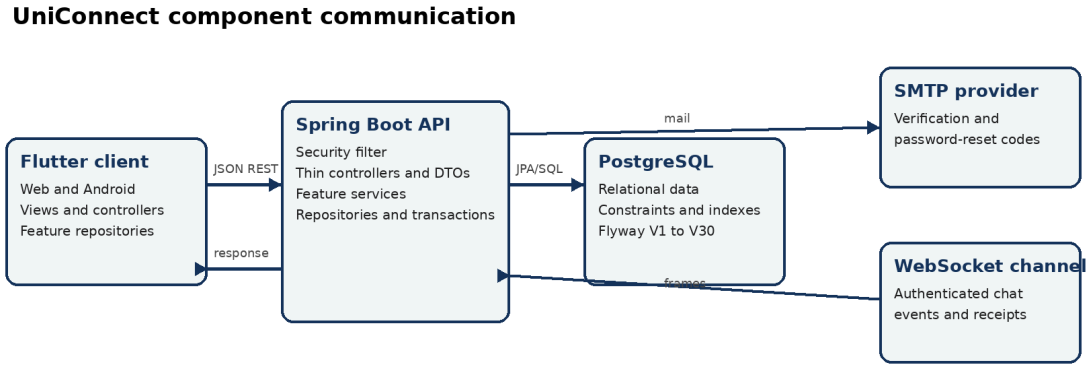


Figure 1 Component communication

Data Model and Database Schema

We modeled users, profiles, club assignments, connections, projects, mentorship sessions, chat, notifications, and reports in our shared domain and class diagrams. We defined the persisted schema through SQL and Java Flyway migrations in code/backend/src/main/resources/db/migration and code/backend/src/main/java/db/migration. We use foreign keys, unique constraints, and service transactions to protect ownership and lifecycle rules. The editable diagrams are in docs/diagrams/UniConnect_Shared_Domain_Model.drawio and docs/diagrams/UniConnect_Overall_Class_Diagram.drawio.

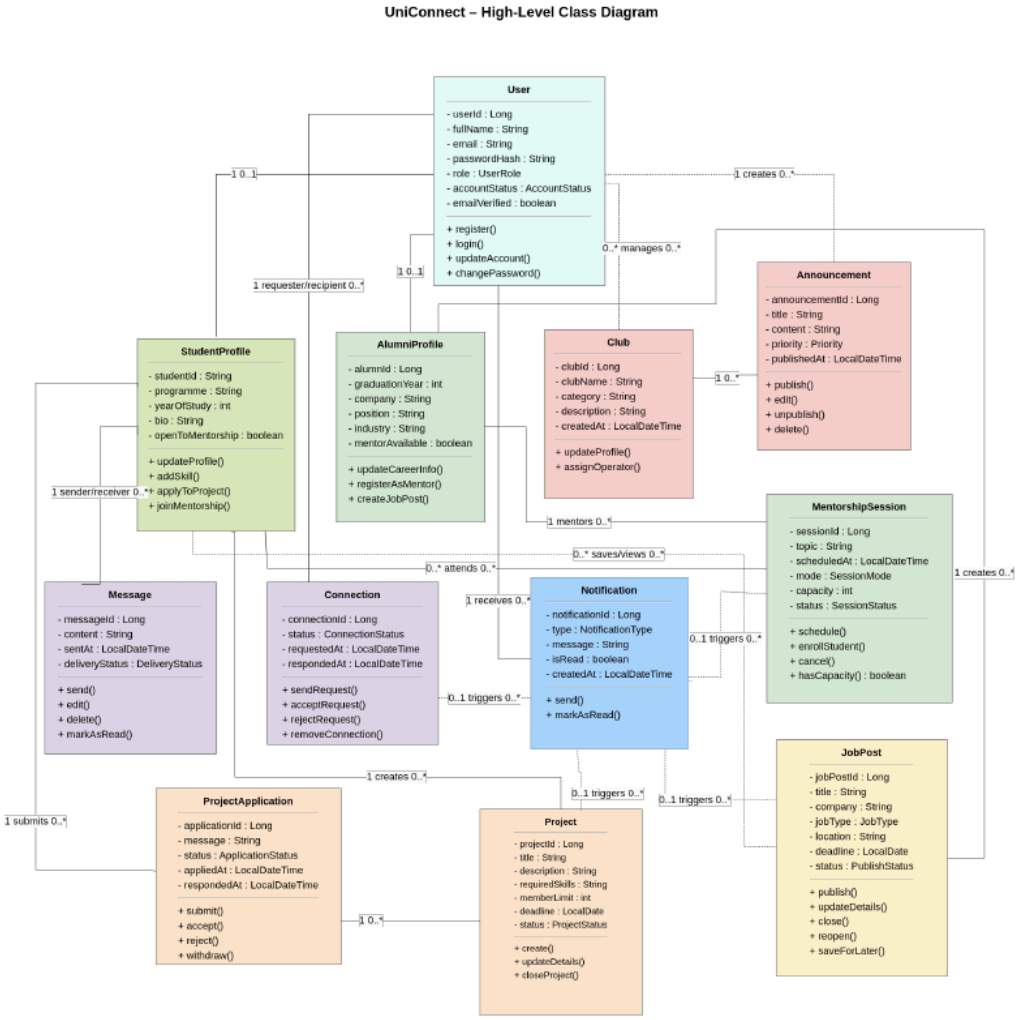


Figure 2 Design-session high-level class model from the submitted Design Report; later implementation contracts differ

Model relationship	Requirement rationale
Account to profile and session	Trusted role-specific identity and revocable access
User to connection, project, and mentorship records	Collaboration with ownership and capacity rules
Club to assignment, announcement, and event	Scoped operator authority
Content to report and retained evidence	Reviewable moderation lifecycle

Design Alternatives

Decision	Selected design	Plausible alternative	Reason and limitation
Backend structure	Feature-based modular monolith	Independent microservices	One deployable unit reduced operational cost and cross-service failure modes for the academic schedule. The trade-off is weaker independent scaling and deployment.
Client	Flutter shared web and Android codebase	Separate web and native Android clients	Shared models, repositories, widgets, and tests reduced duplicate feature work. Large declarative widget classes remain a maintainability risk.
Persistence	PostgreSQL relational schema	Document database	Transactions, constraints, joins, scoped ownership, and audit histories fit relational modelling. Schema evolution requires disciplined migrations.
Realtime chat	REST bootstrap plus WebSocket events	HTTP polling	A persistent bidirectional channel reduces polling overhead. Multi-instance event coordination remains future infrastructure work.

Traceability to the SRS

We aligned feature packages with the SRS groups for authentication, profiles, discovery, connections, clubs, projects, mentorship, chat, careers, notifications, reporting, and administration. We changed older link-based verification and generic removal contracts when later migrations and API work introduced more explicit lifecycles.

Design session feedback

In the Design session, we learned the MVC architecture and used it to rebuild the project as a modular application. We kept our earlier progress in the legacy/pre-mvc-rebuild branch and legacy-v1-final tag. Commit 862bf0c records the rebuild. We do not have a signed session feedback log in the repository.

CO3 Self Assessment

We mapped SRS requirement groups to cohesive feature modules and kept authorization and transactions on the server. We compared plausible alternatives against development time, deployment complexity, data integrity, scalability, and maintenance cost. We cannot claim that the Design session formally approved this exact alternatives table.

6 Implementation

Project Structure

We keep the current application source under code and separate technical documentation from university submission artifacts. The tree below omits generated and build directories.

```
SPL-II_UniConnect/
├── code/
│   ├── backend/src/main/java/com/uniconnect/
│   │   ├── administration  authentication  career  chat  club
│   │   ├── connection  discovery  mentorship  notification
│   │   ├── profile  project  reporting  shared
│   │   └── db/migration
│   └── frontend/lib/
│       ├── core/{config,network,routing,session,theme,widgets}
│       └── features/{administration,authentication,career,chat,club,
│                     connection,home,mentorship,notification,profile,
│                     project,reporting}
├── doc/                    university submission artifacts
├── development-docs/       technical documentation and evidence
├── scripts/                repeatable local/demo tooling
└── .github/workflows/      continuous integration
```

Communication and Data Boundaries

We let Flutter views delegate to controllers or local view states, then call feature repositories. Repositories use our shared authenticated HTTP client and map JSON into Dart models. SessionAuthenticationFilter checks protected requests against persisted sessions. Spring controllers validate request DTOs and call services; services enforce business rules, ownership,

privacy, and transaction boundaries before using repositories. We serialize response DTOs as JSON for Flutter.

Representative end-to-end request flow



Figure 3 Registration and verification flow

Step	Evidence and responsibility
Flutter input	sign_up_page.dart and RegistrationController validate fields and call RegistrationRepository.
HTTP contract	POST /api/v1/auth/register returns the existing registration response; POST /api/v1/auth/verify-email accepts email and code and returns 204.
Security boundary	SecurityConfig permits only the public authentication routes; protected routes require a valid database-backed session.
Registration service	RegistrationService enforces university domain, role, duplicate-email, password policy, BCrypt encoding, and pending-account creation.
Challenge service	EmailVerificationService generates a secure six-digit string, stores only a salted account-bound digest, controls attempts and resend, and activates atomically.
Persistence	JPA repositories and Flyway V27 implement challenge state, limits, constraints, and legacy-link invalidation.

Division of Classes and Methods

We split backend responsibilities among controllers for HTTP handling, DTOs for API boundaries, services for use cases and transactions, domain entities for state and invariants, and repositories for persistence. We use constructor injection to expose dependencies and simplify isolated tests. In Flutter, we group models, repositories, controllers, views, and reusable widgets by feature, while core packages hold transport, routing, session, theming, and pagination code.

Language	Classes	Average class LOC	Maximum class LOC	Methods	Average method LOC	Maximum method LOC
Java	195	54.75	757	1,304	5.73	57
Dart	416	72.75	617	1,135	23.67	287

We counted physical nonblank, noncomment lines inside lexically matched types and brace-bodied methods. We excluded tests, generated and built output, scripts, and the archived mobile prototype. We counted nested types separately while they still contribute to enclosing type spans. Because our scanner is lexical rather than a Java or Dart parser, its method population omits Dart arrow functions and abstract declarations.
Our working-tree scan found MentorshipSlotService as the largest Java class at 757 lexical lines and MentorshipController as the largest Dart class at 617. It found V26__correct_mentorship_lifecycle.migrate as the largest Java method at 57 lines and _MyMentorshipTabState.build as the largest Dart method at 287. We should split large Flutter build methods where that improves review and test isolation.

Implemented Features

Area	Status	Implemented behavior
Authentication and sessions	Implemented	Registration, code verification, login, refresh, logout, recovery, role/context eligibility, and session revocation.
Profiles and networking	Implemented	Student/alumni profiles, privacy, discovery, requests, acceptance, rejection, cancellation, removal, and connection views.
Clubs and publications	Implemented	Creation requests, assignments, contexts, announcements, events, and administrative lifecycles.

Projects	Implemented	Posts, vacancies, recommendation scoring, applications, decisions, membership, capacity, and status.
Mentorship	Implemented	Mentor eligibility, search, finite occurrences, capacity, enrollment, leave/rejoin, closure, cancellation, and privacy.
Chat	Implemented	Accepted-connection eligibility, conversations, messages, history, delivery/read state, WebSocket events, and deletion scopes.
Career	Implemented	Alumni posts, filters, saves, external application links, editing, expiry, and closure.
Administration and reporting	Implemented	Account and club administration, report lifecycle and evidence.

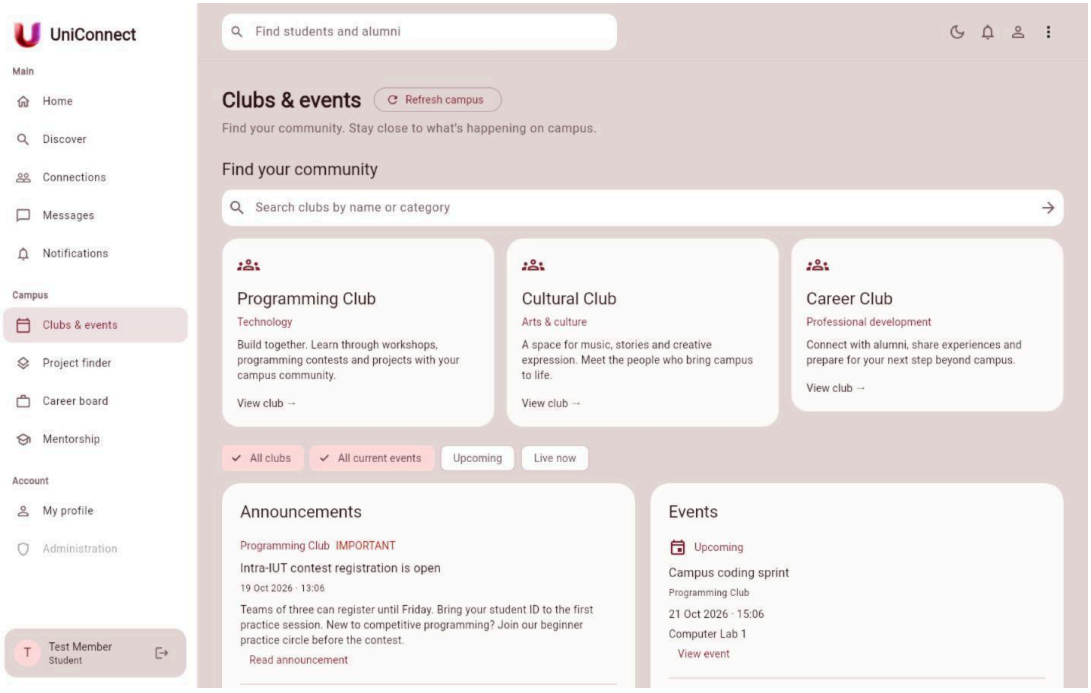


Figure 4 Club interface screenshot from repository evidence

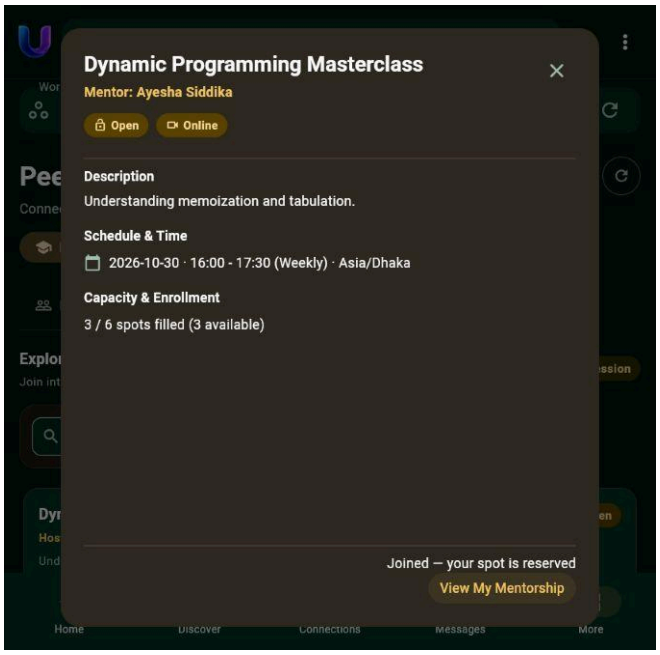


Figure 5 Mentorship joined state from repository QA evidence

Evolution and Refactoring

We established the modular MVC rebuild in commit 862bf0c and moved the canonical source under code in commit c47ebae.
We replaced link verification with numeric codes in PR 100 and V27, then applied the same interaction model to password recovery in V28.
Commit evidence: /c31ca58 ; /39f31e0 ; /c8714c7 .
We corrected mentorship scheduling and enrollment lifecycles in PR 91, then fixed a PostgreSQL filter issue in PR 93.
We removed a people-discovery query explosion in PR 104 and improved chat loading and optimistic send reconciliation in PR 113.
We developed Flutter into a shared web and Android client with three appearance modes and a post-login gateway.

CO1 and CO3 Self Assessment

We demonstrate the concepts from Section 2 through typed DTOs, relational constraints, transactional state changes, authorization services, pagination, and recommendation queries.
We followed the design's feature-oriented three-tier structure while changing verification, recovery, and lifecycle contracts where implementation evidence supported the change.

7 Tools Version Control Testing and Continuous Integration

Technology and Tool Choices

Tool	Use	Reason	Limitation
Flutter and Dart	Shared web and Android client	One feature codebase, responsive widgets, and a mature test runner	Platform-specific behavior and large widget trees still require care.
Spring Boot and Java 21	REST, security, validation, transactions, mail, WebSocket	Integrated dependency injection and mature data/security libraries	Framework upgrades and runtime footprint require maintenance.
PostgreSQL	Durable relational persistence	Transactions, constraints, indexes, and expressive SQL	Network latency and provider operations affect runtime behavior.
Flyway	Ordered schema evolution	Versioned, reviewable, repeatable migrations	Applied migrations cannot be edited safely; upgrades need compatibility tests.
GitHub Actions	Pull-request and main verification	Reproducible backend and shared-client checks	Current workflow is verification, not a production deployment pipeline.

Version Control

We counted 598 commits reachable from local HEAD 341b25b. We used feature branches and pull requests, and many commit subjects reference issues or PRs. We preserved a legacy pre-rebuild branch. We consolidated known Git identity aliases for the four team members, but we do not treat commit counts as measures of effort or sole authorship.

Team member and Git identity	Reachable commits	Evidence note
Saika Sarara / Saika_Sarara	197	Repository attribution; substantial Flutter and chat history.
Sayma Tasnim / SayTas	180	Repository attribution; broad Flutter feature history.

Maliha Tasnim Khan / tasnim240	104	Repository attribution; mentorship-heavy history.
Ayman Binta Altaf Nondiny / aymannondiny	117	Repository attribution; backend, foundation, performance, release, and integration history.

Testing and Continuous Integration

Check	Evidence in repository	Limit
Backend Maven verification	Configured in CI workflow; Java tests present under code/backend/src/test	No current successful run log was available for this report.
Flutter format and analysis	Configured in CI workflow	A workflow definition does not prove a current pass.
Flutter tests	Dart tests present under code/frontend/test and configured in CI	No verified current test total or coverage percentage.
Flutter builds	Web release and Android debug builds configured in CI	No production deployment evidence.
Local development evidence	Presentation seed report documents targeted SQL and API checks	Those checks cover demo data, not every application path.

We configured https://github.com/aymannondiny/SPL-II_UniConnect/blob/341b25b/.github/workflows/ci.yml to verify the backend, pin and install Flutter, check formatting, analyze, run tests, and build web and Android artifacts. We have not documented a production deployment job, coverage percentage, load test, penetration test, or complete accessibility audit.

CO4 Self Assessment

We chose tools that support our cross-platform client, transactional backend, and feature modules, and we list their limits above. We use Git, pull requests, automated tests, and CI. We still need production deployment automation and broader non-functional testing.

8 Professional Ethical and Legal Considerations

Implemented Controls

We restrict registration to the university domain and require verification to reduce unauthorized access.
We hash passwords with BCrypt and keep verification and recovery codes out of API responses and plaintext storage.

We persist sessions so we can revoke access after suspension or sensitive account changes.
We enforce role, context, assignment, ownership, privacy, and connection checks in backend services rather than relying on hidden UI controls.
We use generic public responses to reduce account enumeration and sanitize provider and infrastructure details in API errors.
We retain target snapshots in reports and validate moderation evidence before resolving a report as ACTION_TAKEN.

Outstanding Governance

We have not published a privacy policy, terms of use, retention schedule, consent and withdrawal procedure, data-protection impact assessment, incident-response plan, moderation appeal policy, accessibility report, bias evaluation, or institutional deployment approval. We also need a complete dependency-license inventory and software bill of materials; our root repository has no project license. We have to address these before deployment beyond an academic demonstration.

CO4 Self Assessment

We implemented concrete privacy and security controls, but we cannot infer legal or ethical compliance from code alone. Before real deployment, we will try to integrate license, privacy, retention, accessibility, moderation, and institutional reviews.

9 Project Management and Feasibility

Timeline Evidence

Date	Milestone	Evidence
10 May 2026	Project proposal	Submitted proposal report and planned timeline.
27 July 2026	Combined SRS and design	Submitted design and requirements report.
18 September 2026	Modular MVC rebuild	Commit 862bf0c.
20 to 26 September 2026	Repository, CI, and security foundation	Commits c47ebae, c47860d, and efef46a.
October 2026	Feature completion, lifecycle corrections, presentation data, and UI work	Merged feature and correction pull requests.
27 to 28 October 2026	Verification, discovery, and chat performance changes	PR 100, PR 104, and PR 113.

We delivered the project iteratively through branches, issues, pull requests, CI, and dated delivery notes. We have not recorded reliable planned-versus-actual hours, a cost ledger, sprint

burndown, meeting minutes, risk register, or signed supervisor review in this repository. Git history shows our milestone sequence but cannot prove equal workload or schedule adherence.

Feasibility

We completed the modular monolith and shared Flutter client within the observed repository timeline by limiting deployment units and avoiding duplicate client features. We can use managed PostgreSQL and SMTP providers, but we still need to plan for quotas, network latency, backups, monitoring, and mail delivery. We demonstrated technical feasibility in a controlled setting; we have not measured production capacity or cost.

CO7 Self Assessment

We used Git history, issue tracker and CI to review progress, and our implemented system demonstrates technical feasibility.
When our team faced any scope change, we tried to find out how much of our work was affected by the change and how we can fix the scope without breaking the existing codes. In case of any meeting schedule change we tried to be understanding and have our own works done in the meantime.
We still need to add actual milestone deviations, effort estimates, infrastructure costs, task-tracking screenshots, and scope decisions before claiming complete CO7 evidence.

10 Teamwork and Individual Contributions

We named four team members in the submitted documents. Our Git history shows their identities and topic clusters below.

Team member	Repository-supported contribution areas	Issue assigned
Maliha Tasnim Khan 127- git commits	Mentorship-focused commits and related implementation history; 104 reachable commits.	Assigned tasks, PR links, integration coordination, review work, and qualitative teamwork account.
Sayma Tasnim 139 - git commits	Broad Flutter features history and designing; 180 reachable commits under SayTas.	Assigned screens and repositories, PR links, integration coordination, and qualitative teamwork account.
Ayman Binta Altaf Nondiny 141 - git commits	Backend, foundation, authentication, performance, release, documentation, and integration history; 117 reachable commits.	Exact module ownership, PR links, review work, deployment/presentation duties, and qualitative teamwork account.

Saika Sarara 159 - git commits	Flutter and chat-heavy history; 197 reachable commits under Saika_Sarara.	Assigned screens and chat work, PR links, integration coordination, and qualitative teamwork account.

CO5 Self Assessment

We can show participation by all four members and visible integration points in version control. Every member was responsible for their individual work and can explain those.
In the planning and designing phase, we had fair participation and contribution from all our members. But during the implementation phase, we had to distribute work between the three of us as one of our members was going through a hard-time due to the loss of her father.
We used issue tracker in GitHub and the features were assigned to each member through it. Each member finished their individual work and after every module we tested them by integrating with other complete features.

11 Results Limitations and Future Work

Final Requirement Status

Classification	Requirements	Result
Implemented	Most groups across FR 1 to FR 90	Working backend, Flutter path, and relevant automated evidence for authentication, profiles, discovery, clubs, projects, mentorship, chat, careers, notifications, reporting, and administration.
Implemented with changed contract	FR 2, FR 4, FR 73	Numeric email codes replace older link wording for verification and recovery.
Partial	FR 78	Destination modules re-authorize and refetch targets, but no universal notification-target resolver exists.
Not complete	FR 62, FR 66, FR 69	No global cross-module search, complete irreversible anonymization workflow, or dedicated administrator-warning workflow.

Known Limitations

We have not added real-time presence or shared WebSocket events across multiple backend instances.
We have not committed production-scale load, availability, accessibility, security, or user-satisfaction studies.
We have not documented production secrets, monitoring, backups, recovery drills, or incident response in the repository.
We still need to split several large Flutter state and build methods where that improves readability and test isolation.

Future Work

We plan to complete deferred requirement groups, run PostgreSQL-specific tests in CI, evaluate accessibility and usability, establish dependency and privacy governance, improve observability and backups, coordinate WebSocket events across backend instances, and refactor the largest methods and classes. We have not committed a schedule for this work.

CO6 Self Assessment

We distinguish completed, changed, partial, and incomplete requirements. We connect the problem, requirements, design, implementation, tests, and remaining work without claiming production readiness. So an informed reader should understand the problem, requirements, design decisions, implementation status, and limitations from the report and project documents.

12 Glossary and References

Glossary

Term	Meaning in UniConnect
Active context	The personal, club, or system-administration operating context selected inside an authenticated session. It does not independently grant authority.
DTO	Data Transfer Object used as an API request or response boundary.
Flyway migration	An ordered, one-time database schema or data change recorded in migration history.
Platform role	Persistent STUDENT, ALUMNI, or SYSTEM_ADMIN classification.
Scoped assignment	A current authorization link between a user, a club, and a club role.

Session	A persisted access and refresh record that the backend revalidates and can revoke.
Verification challenge	An account and email-bound salted digest representing a six-digit code; the plaintext code is delivered but not persisted.
WebSocket	A bidirectional connection used for authenticated chat events after REST bootstrap.

CO6 Self Assessment

Our document started with a simple explanation of our project and ended with the interpretation of the jargon and acronyms used. Hence even if the document is mainly targeted for the evaluators who will judge our project, any non-technical person can read it and get a decent idea about how our app UniConnect works.
On the other hand, the presentation slides are pretty straight-forward and easy to be understood by anyone just by going through them.

References

[1] UniConnect Team 4, SPL-II Project Proposal Report, 10 May 2026, Presentation&Report/SPL-2_Team4_ProjectProposalReport.pdf.
[2] UniConnect Team 4, Design Report, 27 July 2026, Presentation&Report/SPL-2_Team-4_Design Report.pdf.
[3] UniConnect Team 4, Requirement Collection Document, Presentation&Report/Group-4_SPL-II Requirement Collection Document .xlsx.
[4] Flutter documentation, Build for and integrate with multiple platforms, https://docs.flutter.dev/platform-integration.
[5] Spring Boot reference documentation, https://docs.spring.io/spring-boot/reference/index.html.
[6] I. Fette and A. Melnikov, The WebSocket Protocol, RFC 6455, December 2011, https://www.rfc-editor.org/info/rfc6455/.
[7] Redgate, Flyway migration documentation, https://documentation.red-gate.com/flyway/reference/commands/migrate.
[8] Meta, Sharing Our Vision for Communities on WhatsApp, 2022, https://about.fb.com/news/2022/04/our-vision-for-communities-on-whatsapp/.
[9] LinkedIn Help, Filter and sort job search results, https://www.linkedin.com/help/linkedin/answer/a507441/.